

FILE Engine

- [1. Kompakte System-Referenz für KI-Modelle](#)
- [2. Operationen](#)
- [3. EXEC – Dateioperationen](#)
- [4. READ – Datei lesen](#)
 - [4.1 Parameter](#)
- [5. WRITE – Datei schreiben](#)
 - [5.1 Parameter](#)
- [6. APPEND – Text anhängen](#)
 - [6.1 Parameter](#)
- [7. COPY – Datei kopieren](#)
 - [7.1 Parameter](#)
- [8. MOVE – Datei verschieben](#)
 - [8.1 Parameter](#)
- [9. DELETE – Datei löschen](#)
 - [9.1 Parameter](#)
- [10. EXISTS – Existenz prüfen und speichern](#)
- [11. Relative und absolute Pfade](#)
- [12. FILE Engine und OC Engine](#)
- [13. BUFFER – Datenextraktion](#)
- [14. BUFFER Actions](#)
 - [14.1 TEXT](#)
 - [14.2 FILTER](#)
 - [14.3 LINE](#)
 - [14.4 LAST_LINE](#)
 - [14.5 COUNT](#)
- [15. ASSERT – Validierung](#)
- [16. Filesystem Assertions](#)
 - [EXISTS](#)
 - [NOT_EXISTS](#)
- [17. Content Assertions](#)
 - [17.1 CONTAINS](#)
 - [17.2 NOT_CONTAINS](#)
 - [17.3 EQUALS](#)
 - [17.4 NOT_EQUALS](#)
 - [17.5 COUNT](#)
- [18. EXISTS: EXEC vs. ASSERT](#)
 - [EXEC EXISTS](#)
 - [ASSERT EXISTS](#)
- [19. Platzhalter](#)
- [20. Vollständiges Beispiel](#)
- [21. Troubleshooting & Best Practices](#)
 - [Datei wird nicht gefunden](#)
 - [Relative Pfade](#)
 - [Cleanup](#)
 - [BUFFER benötigt READ](#)
 - [Content ASSERT benötigt response](#)
- [22. Kurzreferenz](#)
- [23. Design-Grundsatz](#)

Die **FILE Engine** ermöglicht Dateioperationen auf dem lokalen Dateisystem des AGATE-Prozesses.

Sie unterstützt:

- Lesen von Dateien
- Schreiben und Überschreiben von Dateien
- Anhängen von Text
- Kopieren und Verschieben von Dateien
- Löschen von Dateien
- Prüfen der Dateixistenz
- Extraktion von Dateiinhalten über **BUFFER**
- Validierung von Dateiinhalten und Dateixistenz über **ASSERT**

Die FILE Engine arbeitet ausschließlich auf dem **lokalen Dateisystem des AGATE-Prozesses**.

1. Kompakte System-Referenz für KI-Modelle

```
1 # AGATE FILE-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 # READ
5 - type: FILE
6   op: EXEC
7   action: READ
8   path: "...
9   response: "...
10  encoding: "UTF-8"      # Optional, Default: UTF-8
11  condition: "...      # Optional
12
13 # WRITE
14 - type: FILE
15   op: EXEC
16   action: WRITE
17   path: "...
18   text: "...
19   encoding: "UTF-8"    # Optional, Default: UTF-8
20   overwrite: true      # Optional
21   condition: "...
22
23 # APPEND
24 - type: FILE
25   op: EXEC
26   action: APPEND
27   path: "...
28   text: "...
29   encoding: "UTF-8"    # Optional, Default: UTF-8
30   newline: false       # Optional, Default: false
31   condition: "...
32
33 # COPY
34 - type: FILE
35   op: EXEC
36   action: COPY
37   source: "...
```

```

38 target: "...
39 overwrite: false          # Optional
40 condition: "...
41
42 # MOVE
43 - type: FILE
44   op: EXEC
45   action: MOVE
46   source: "...
47   target: "...
48   overwrite: false        # Optional
49   condition: "...
50
51 # DELETE
52 - type: FILE
53   op: EXEC
54   action: DELETE
55   path: "...
56   missingOk: false        # Optional, Default: false
57   condition: "...
58
59 # EXISTS
60 - type: FILE
61   op: EXEC
62   action: EXISTS
63   path: "...
64   response: "...
65   condition: "...
66
67 # BUFFER
68 - type: FILE
69   op: BUFFER
70   response: "...
71   action: <TEXT|FILTER|LINE|LAST_LINE|COUNT>
72   value: "...
73   name: "...
74   condition: "...
75
76 # ASSERT
77 - type: FILE
78   op: ASSERT
79   action:
80 <EXISTS|NOT_EXISTS|CONTAINS|NOT_CONTAINS|EQUALS|NOT_EQUALS|COUNT>
81   path: "...
82   response: "...
83   value: "...
84   expected: "...
85   condition: "...

```

2. Operationen

Die FILE Engine verwendet dieselbe Grundstruktur wie andere AGATE Engines:

```

1 | type   = Engine
2 | op     = Ausführungsmodus
3 | action = konkrete Dateioperation
4 |

```

Beispiel:

```

1 - type: FILE
2   op: EXEC

```

```
3 action: READ
4
```

Die Engine unterstützt drei `op`-Varianten:

Operation	Beschreibung
<code>EXEC</code>	Führt eine Dateioperation aus.
<code>BUFFER</code>	Extrahiert Informationen aus einem zuvor gelesenen Dateiinhalt.
<code>ASSERT</code>	Validiert Dateiexistenz oder Dateiinhalt.

3. EXEC – Dateioperationen

`EXEC` führt eine konkrete Dateioperation aus.

Unterstützte Actions:

```
1 READ
2 WRITE
3 APPEND
4 COPY
5 MOVE
6 DELETE
7 EXISTS
8
```

4. READ – Datei lesen

`READ` liest den vollständigen Inhalt einer Datei und speichert ihn unter `response`.

```
1 - type: FILE
2   op: EXEC
3   action: READ
4   path: "output/report.txt"
5   response: report
6
```

Der gespeicherte Inhalt kann anschließend mit `BUFFER` oder `ASSERT` weiterverarbeitet werden.

4.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
-----------	--------------	--------------	---------

path	Pfad der zu lesenden Datei	Ja	–
response	Name für den gelesenen Dateiinhalt	Ja	–
encoding	Zeichencodierung	Nein	UTF-8
condition	Bedingung für die Ausführung	Nein	–

Beispiel mit explizitem Encoding:

```

1 - type: FILE
2   op: EXEC
3   action: READ
4   path: "output/report.txt"
5   response: report
6   encoding: UTF-8
7

```

5. WRITE – Datei schreiben

WRITE schreibt Text in eine Datei.

```

1 - type: FILE
2   op: EXEC
3   action: WRITE
4   path: "output/report.txt"
5   text: "AGATE FILE Engine"
6

```

Mehrzeilige Inhalte können direkt über YAML angegeben werden:

```

1 - type: FILE
2   op: EXEC
3   action: WRITE
4   path: "output/report.txt"
5   text: |
6     AGATE FILE Engine
7     Line 2
8     Line 3
9

```

5.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
path	Zielpfad	Ja	–

text	Zu schreibender Inhalt	Ja	–
encoding	Zeichencodierung	Nein	UTF-8
overwrite	Vorhandene Datei überschreiben	Nein	Engine-Default
condition	Bedingung für die Ausführung	Nein	–

Beispiel:

```

1 - type: FILE
2   op: EXEC
3   action: WRITE
4   path: "output/report.txt"
5   text: "{B[report_content]}"
6   overwrite: true
7

```

6. APPEND – Text anhängen

APPEND fügt Text an eine bestehende Datei an.

```

1 - type: FILE
2   op: EXEC
3   action: APPEND
4   path: "output/report.txt"
5   text: "additional line"
6

```

Standardmäßig wird kein zusätzlicher Zeilenumbruch eingefügt.

```

1 newline = false
2

```

Soll nach dem angehängten Text ein Zeilenumbruch erzeugt werden:

```

1 - type: FILE
2   op: EXEC
3   action: APPEND
4   path: "output/report.txt"
5   text: "additional line"
6   newline: true
7

```

6.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
-----------	--------------	--------------	---------

path	Zieldatei	Ja	–
text	Anzuhängender Text	Ja	–
encoding	Zeichencodierung	Nein	UTF-8
newline	Zeilenumbruch anhängen	Nein	false
condition	Bedingung für die Ausführung	Nein	–

7. COPY – Datei kopieren

COPY kopiert eine Datei von source nach target.

```

1 - type: FILE
2   op: EXEC
3   action: COPY
4   source: "output/report.txt"
5   target: "backup/report.txt"
6
```

Soll eine vorhandene Zieldatei ersetzt werden:

```

1 - type: FILE
2   op: EXEC
3   action: COPY
4   source: "output/report.txt"
5   target: "backup/report.txt"
6   overwrite: true
7
```

7.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
source	Quelldatei	Ja	–
target	Zieldatei	Ja	–
overwrite	Vorhandene Zieldatei überschreiben	Nein	false

condition	Bedingung für die Ausführung	Nein	–
-----------	------------------------------	------	---

8. MOVE – Datei verschieben

MOVE verschiebt eine Datei von source nach target .

```

1 - type: FILE
2   op: EXEC
3   action: MOVE
4   source: "output/report.txt"
5   target: "archive/report.txt"
6
```

MOVE kann auch zum Umbenennen verwendet werden.

Beispiel:

```

1 - type: FILE
2   op: EXEC
3   action: MOVE
4   source: "output/report.txt"
5   target: "output/report-old.txt"
6
```

Dadurch ist keine separate RENAME -Action erforderlich.

8.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
source	Quelldatei	Ja	–
target	Zielpfad	Ja	–
overwrite	Vorhandenes Ziel überschreiben	Nein	false
condition	Bedingung für die Ausführung	Nein	–

9. DELETE – Datei löschen

DELETE löscht eine Datei.

```

1 - type: FILE
2   op: EXEC
3   action: DELETE
```

```
4 path: "output/report.txt"
5
```

Standardmäßig schlägt der Schritt fehl, wenn die Datei nicht existiert.

```
1 missingOk = false
2
```

Falls eine nicht vorhandene Datei akzeptiert werden soll:

```
1 - type: FILE
2   op: EXEC
3   action: DELETE
4   path: "output/report.txt"
5   missingOk: true
6
```

Dies ist besonders für Cleanup-Schritte sinnvoll.

9.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
path	Zu löschende Datei	Ja	–
missingOk	Fehlende Datei als Erfolg behandeln	Nein	false
condition	Bedingung für die Ausführung	Nein	–

10. EXISTS – Existenz prüfen und speichern

`EXISTS` prüft, ob eine Datei existiert, und speichert das Ergebnis als Boolean unter `response`.

```
1 - type: FILE
2   op: EXEC
3   action: EXISTS
4   path: "output/report.txt"
5   response: report_exists
6
```

Anschließend kann das Ergebnis verwendet werden:

```
1 {B[report_exists]}
2
```

Beispielsweise in einer Condition:

```
1 - type: FILE
2   condition: "'{B[report_exists]}' == 'true'"
3   op: EXEC
4   action: READ
5   path: "output/report.txt"
6   response: report
7
```

Wichtig:

```
1 FILE EXEC / EXISTS
2
```

erzeugt einen Wert für die weitere Verarbeitung.

Das ist etwas anderes als:

```
1 FILE ASSERT / EXISTS
2
```

welches direkt eine Prüfung durchführt.

11. Relative und absolute Pfade

Die FILE Engine arbeitet auf dem lokalen Dateisystem des AGATE-Prozesses.

Absolute Pfade werden unverändert verwendet.

Beispiel:

```
1 path: "C:\\tmp\\report.txt"
2
```

Relative Pfade werden relativ zum Arbeitsverzeichnis des AGATE-Prozesses aufgelöst.

Beispiel:

```
1 path: "output/report.txt"
2
```

Wenn AGATE beispielsweise aus folgendem Verzeichnis gestartet wurde:

```
1 C:\work\projects\agate-server
2
```

entspricht der relative Pfad:

```
1 output/report.txt
2
```

dem effektiven Pfad:

```
1 C:\work\projects\agate-server\output\report.txt
2
```

Dasselbe gilt für:

```
1 path
2 source
3 target
4
```

12. FILE Engine und OC Engine

Die FILE Engine arbeitet immer lokal.

```
1 FILE
2   -> lokales Dateisystem des AGATE-Prozesses
3
```

Dateien innerhalb eines OpenShift-Pods werden über die OC Engine verarbeitet.

Beispiel:

```
1 FILE READ
2   -> lokale Datei lesen
3
4 OC EXEC cat ...
5   -> Datei im Pod lesen
6
7 OC GET
8   -> Datei aus Pod nach lokal kopieren
9
10 OC PUT
11   -> lokale Datei in Pod kopieren
12
```

Damit bleibt die Verantwortlichkeit klar getrennt.

13. BUFFER – Datenextraktion

BUFFER arbeitet auf dem Inhalt eines vorherigen **FILE EXEC / READ**.

Beispiel:

```
1 - type: FILE
2   op: EXEC
3   action: READ
4   path: "output/report.txt"
5   response: report_raw
6
7 - type: FILE
8   op: BUFFER
9   response: report_raw
10  action: TEXT
11  name: report
12
```

Wichtig:

BUFFER liest nicht direkt erneut von einem Dateipfad.

Es arbeitet auf dem unter **response** gespeicherten Dateinhalt.

14. BUFFER Actions

Unterstützte Actions:

```
1 TEXT
2 FILTER
3 LINE
4 LAST_LINE
5 COUNT
6
```

Action	Beschreibung	Parameter
TEXT	Speichert den kompletten getrimmten Dateiinhalt	–
FILTER	Speichert Zeilen, die einen Suchbegriff enthalten	value
LINE	Extrahiert eine Zeile; 0 = erste Zeile	value optional
LAST_LINE	Extrahiert eine Zeile relativ zum Dateiende	value optional
COUNT	Zählt Vorkommen eines Suchbegriffs	value

14.1 TEXT

```
1 - type: FILE
2   op: BUFFER
3   response: report_raw
4   action: TEXT
5   name: report
6
```

14.2 FILTER

```
1 - type: FILE
2   op: BUFFER
3   response: report_raw
4   action: FILTER
5   value: "ERROR"
6   name: error_lines
7
```

Es werden nur Zeilen gespeichert, die den Suchbegriff enthalten.

14.3 LINE

```
1 - type: FILE
2   op: BUFFER
3   response: report_raw
4   action: LINE
5   value: 0
6   name: first_line
7
```

Dabei gilt:

```
1 0 = erste Zeile
2 1 = zweite Zeile
3 2 = dritte Zeile
4 ...
5
```

Wird `value` nicht angegeben, wird `0` verwendet.

14.4 LAST_LINE

```
1 - type: FILE
2   op: BUFFER
3   response: report_raw
4   action: LAST_LINE
5   value: 0
6   name: last_line
7
```

Dabei gilt:

```
1 0 = letzte Zeile
2 1 = vorletzte Zeile
3 2 = drittletzte Zeile
4 ...
5
```

14.5 COUNT

```
1 - type: FILE
2   op: BUFFER
3   response: report_raw
4   action: COUNT
5   value: "CHANGED"
6   name: changed_count
7
```

Das Ergebnis steht anschließend über:

```
1 {B[changed_count]}
2
```

zur Verfügung.

15. ASSERT – Validierung

Die FILE Engine unterstützt zwei Arten von Assertions:

- ```
1 | 1. Filesystem Assertions
2 | 2. Content Assertions
3 |
```

---

## 16. Filesystem Assertions

Filesystem Assertions arbeiten direkt mit einem Pfad.

Unterstützte Actions:

- ```
1 | EXISTS
2 | NOT_EXISTS
3 |
```

EXISTS

- ```
1 | - type: FILE
2 | op: ASSERT
3 | action: EXISTS
4 | path: "output/report.txt"
5 |
```

Der Schritt ist erfolgreich, wenn die Datei existiert.

### NOT\_EXISTS

- ```
1 | - type: FILE
2 |   op: ASSERT
3 |   action: NOT_EXISTS
4 |   path: "output/report.txt"
5 |
```

Der Schritt ist erfolgreich, wenn die Datei nicht existiert.

17. Content Assertions

Content Assertions arbeiten auf dem `response` eines vorherigen `READ`.

Unterstützte Actions:

- ```
1 | CONTAINS
2 | NOT_CONTAINS
3 | EQUALS
4 | NOT_EQUALS
5 | COUNT
6 |
```

---

### 17.1 CONTAINS

```
1 - type: FILE
2 op: ASSERT
3 response: report_raw
4 action: CONTAINS
5 value: "CHANGED"
6
```

Prüft, ob der Suchtext im Dateinhalt vorkommt.

---

## 17.2 NOT\_CONTAINS

```
1 - type: FILE
2 op: ASSERT
3 response: report_raw
4 action: NOT_CONTAINS
5 value: "ERROR"
6
```

Prüft, ob der Suchtext nicht vorkommt.

---

## 17.3 EQUALS

```
1 - type: FILE
2 op: ASSERT
3 response: report_raw
4 action: EQUALS
5 value: "AGATE FILE Engine"
6
```

Prüft auf exakte Übereinstimmung nach der für die Engine vorgesehenen Textnormalisierung.

---

## 17.4 NOT\_EQUALS

```
1 - type: FILE
2 op: ASSERT
3 response: report_raw
4 action: NOT_EQUALS
5 value: "unexpected value"
6
```

Prüft, dass der Inhalt nicht dem Vergleichswert entspricht.

---

## 17.5 COUNT

```
1 - type: FILE
2 op: ASSERT
3 response: report_raw
4 action: COUNT
5 value: "CHANGED"
6 expected: 2
7
```

Prüft die Anzahl der Vorkommen eines Suchbegriffs.

---

## 18. EXISTS: EXEC vs. ASSERT

Die FILE Engine unterstützt `EXISTS` bewusst in zwei Varianten.

### EXEC EXISTS

```
1 - type: FILE
2 op: EXEC
3 action: EXISTS
4 path: "output/report.txt"
5 response: exists_result
6
```

Ergebnis:

```
1 true / false
2
```

Das Ergebnis kann später verwendet werden.

### ASSERT EXISTS

```
1 - type: FILE
2 op: ASSERT
3 action: EXISTS
4 path: "output/report.txt"
5
```

Der Testschritt schlägt direkt fehl, wenn die Datei nicht existiert.

Kurz gesagt:

```
1 EXEC EXISTS
2 -> liefert Information
3
4 ASSERT EXISTS
5 -> validiert eine Erwartung
6
```

---

## 19. Platzhalter

Dateipfade und Inhalte können normale AGATE-Platzhalter verwenden.

Beispiel:

```
1 variables:
2 baseDir: "output"
3 filename: "report.txt"
4
5 steps:
6
7 - type: FILE
8 op: EXEC
9 action: WRITE
10 path: "${B[baseDir]}/${B[filename]}"
11 text: "AGATE"
12
13 - type: FILE
14 op: EXEC
```

```
15 action: READ
16 path: "{B[baseDir]}/{B[filename]}"
17 response: report
18
```

Unterstützte Platzhalter folgen den allgemeinen AGATE-Regeln, beispielsweise:

```
1 {B[var]}
2 {E[env.*]}
3 {XL[column]}
4
```

---

## 20. Vollständiges Beispiel

```
1 testCases:
2
3 - id: TC_FILE_FULL_DEMO
4 description: "FILE Engine - Full Happy Path"
5 stage: "*"
6 priority: HIGH
7
8 variables:
9 baseDir: "output/file-demo"
10 filename: "test.txt"
11 copyFilename: "test-copy.txt"
12 movedFilename: "test-moved.txt"
13
14 steps:
15
16 # Datei schreiben
17 - type: FILE
18 op: EXEC
19 action: WRITE
20 path: "{B[baseDir]}/{B[filename]}"
21 text: |
22 AGATE FILE Engine
23 Line 2
24 CHANGED
25 Line 4
26 CHANGED
27 response: write_result
28
29 # Existenz direkt prüfen
30 - type: FILE
31 op: ASSERT
32 action: EXISTS
33 path: "{B[baseDir]}/{B[filename]}"
34
35 # Existenz als Boolean speichern
36 - type: FILE
37 op: EXEC
38 action: EXISTS
39 path: "{B[baseDir]}/{B[filename]}"
40 response: file_exists
41
42 # Datei lesen
43 - type: FILE
44 op: EXEC
45 action: READ
46 path: "{B[baseDir]}/{B[filename]}"
47 response: file_content
48
49 # Inhalt prüfen
```

```

50 - type: FILE
51 op: ASSERT
52 response: file_content
53 action: CONTAINS
54 value: "AGATE FILE Engine"
55
56 - type: FILE
57 op: ASSERT
58 response: file_content
59 action: NOT_CONTAINS
60 value: "ERROR"
61
62 - type: FILE
63 op: ASSERT
64 response: file_content
65 action: COUNT
66 value: "CHANGED"
67 expected: 2
68
69 # Vollständigen Inhalt puffern
70 - type: FILE
71 op: BUFFER
72 response: file_content
73 action: TEXT
74 name: complete_file
75
76 # Gefilterte Zeilen puffern
77 - type: FILE
78 op: BUFFER
79 response: file_content
80 action: FILTER
81 value: "CHANGED"
82 name: changed_lines
83
84 # Erste Zeile
85 - type: FILE
86 op: BUFFER
87 response: file_content
88 action: LINE
89 value: 0
90 name: first_line
91
92 # Letzte Zeile
93 - type: FILE
94 op: BUFFER
95 response: file_content
96 action: LAST_LINE
97 value: 0
98 name: last_line
99
100 # Anzahl
101 - type: FILE
102 op: BUFFER
103 response: file_content
104 action: COUNT
105 value: "CHANGED"
106 name: changed_count
107
108 # Text anhängen
109 - type: FILE
110 op: EXEC
111 action: APPEND
112 path: "{B[baseDir]}/{B[filename]}"
113 text: "APPENDED"
114 newline: true
115
116 # Neu lesen

```

```
117 - type: FILE
118 op: EXEC
119 action: READ
120 path: "{B[baseDir]}/{B[filename]}"
121 response: file_after_append
122
123 - type: FILE
124 op: ASSERT
125 response: file_after_append
126 action: CONTAINS
127 value: "APPENDED"
128
129 # Kopieren
130 - type: FILE
131 op: EXEC
132 action: COPY
133 source: "{B[baseDir]}/{B[filename]}"
134 target: "{B[baseDir]}/{B[copyFilename]}"
135 overwrite: true
136
137 - type: FILE
138 op: ASSERT
139 action: EXISTS
140 path: "{B[baseDir]}/{B[copyFilename]}"
141
142 # Verschieben
143 - type: FILE
144 op: EXEC
145 action: MOVE
146 source: "{B[baseDir]}/{B[copyFilename]}"
147 target: "{B[baseDir]}/{B[movedFilename]}"
148 overwrite: true
149
150 - type: FILE
151 op: ASSERT
152 action: NOT_EXISTS
153 path: "{B[baseDir]}/{B[copyFilename]}"
154
155 - type: FILE
156 op: ASSERT
157 action: EXISTS
158 path: "{B[baseDir]}/{B[movedFilename]}"
159
160 # Löschen
161 - type: FILE
162 op: EXEC
163 action: DELETE
164 path: "{B[baseDir]}/{B[movedFilename]}"
165
166 - type: FILE
167 op: ASSERT
168 action: NOT_EXISTS
169 path: "{B[baseDir]}/{B[movedFilename]}"
170
171 - type: FILE
172 op: EXEC
173 action: DELETE
174 path: "{B[baseDir]}/{B[filename]}"
175
176 # Cleanup darf bereits fehlende Datei akzeptieren
177 - type: FILE
178 op: EXEC
179 action: DELETE
180 path: "{B[baseDir]}/{B[filename]}"
181 missingOk: true
182
```

---

## 21. Troubleshooting & Best Practices

### Datei wird nicht gefunden

Prüfe:

- ob der Pfad korrekt ist,
- ob ein relativer Pfad vom erwarteten AGATE-Arbeitsverzeichnis ausgeht,
- ob vorherige Schritte die Datei tatsächlich erzeugt haben.

Bei Unsicherheit kann zunächst geprüft werden:

```
1 - type: FILE
2 op: ASSERT
3 action: EXISTS
4 path: "output/report.txt"
5
```

---

### Relative Pfade

Für portable Testfälle sollten relative Pfade bevorzugt werden:

```
1 path: "output/report.txt"
2
```

Absolute Pfade wie:

```
1 path: "C:\\tmp\\report.txt"
2
```

sind stärker an die konkrete Ausführungsumgebung gekoppelt.

---

### Cleanup

Für Cleanup-Schritte ist häufig sinnvoll:

```
1 missingOk: true
2
```

Beispiel:

```
1 - type: FILE
2 op: EXEC
3 action: DELETE
4 path: "output/temp.txt"
5 missingOk: true
6
```

Dadurch bleibt ein Cleanup idempotent.

---

### BUFFER benötigt READ

**FILE BUFFER** arbeitet auf einem zuvor gespeicherten Dateiinhalt.

Falsch:

```
1 - type: FILE
2 op: BUFFER
3 path: "output/report.txt"
4 action: TEXT
5 name: report
6
```

Richtig:

```
1 - type: FILE
2 op: EXEC
3 action: READ
4 path: "output/report.txt"
5 response: report_raw
6
7 - type: FILE
8 op: BUFFER
9 response: report_raw
10 action: TEXT
11 name: report
12
```

---

## Content ASSERT benötigt response

Auch Content Assertions arbeiten auf einem zuvor gelesenen Inhalt.

```
1 - type: FILE
2 op: EXEC
3 action: READ
4 path: "output/report.txt"
5 response: report
6
7 - type: FILE
8 op: ASSERT
9 response: report
10 action: CONTAINS
11 value: "SUCCESS"
12
```

Filesystem Assertions verwenden dagegen direkt **path** :

```
1 - type: FILE
2 op: ASSERT
3 action: EXISTS
4 path: "output/report.txt"
5
```

---

## 22. Kurzreferenz

```
1 FILE
2 |
3 | EXEC
4 | |
5 | | READ
6 | | |
7 | | | path
8 | | | response
9 | | | encoding [default: UTF-8]
```

```

7
8 |— WRITE
9 | |— path
10 | |— text
11 | |— encoding [default: UTF-8]
12 | |— overwrite
13
14 |— APPEND
15 | |— path
16 | |— text
17 | |— encoding [default: UTF-8]
18 | |— newline [default: false]
19
20 |— COPY
21 | |— source
22 | |— target
23 | |— overwrite [default: false]
24
25 |— MOVE
26 | |— source
27 | |— target
28 | |— overwrite [default: false]
29
30 |— DELETE
31 | |— path
32 | |— missingOk [default: false]
33
34 |— EXISTS
35 | |— path
36 | |— response
37
38 |— BUFFER
39 | |— TEXT
40 | |— FILTER
41 | |— LINE
42 | |— LAST_LINE
43 | |— COUNT
44
45 |— ASSERT
46 | |— EXISTS
47 | |— NOT_EXISTS
48 | |— CONTAINS
49 | |— NOT_CONTAINS
50 | |— EQUALS
51 | |— NOT_EQUALS
52 | |— COUNT
53

```

## 23. Design-Grundsatz

Die FILE Engine folgt bewusst demselben AGATE-Muster wie CMD und OC:

```

1 EXEC
2 -> Operation ausführen
3
4 BUFFER
5 -> Ergebnis extrahieren
6
7 ASSERT
8 -> Ergebnis validieren
9

```

Dadurch bleibt die DSL über verschiedene Engines hinweg konsistent.

Ein Benutzer muss nicht für jede technische Domäne ein vollständig neues Bedienkonzept lernen.

Die FILE Engine abstrahiert dabei ausschließlich allgemeine, plattformunabhängige Dateioperationen und bildet keine vollständige Betriebssystem-Shell nach.